

Entraîner une IA

Affiner, spécialiser et dialoguer
avec ses propres modèles, sur sa machine

Cours complet pour débutants

Table des matières

0	Avant-propos	1
0.1	À qui s'adresse ce cours	1
0.2	Ce que vous saurez faire à la fin	1
0.3	Comment lire ce cours	1
0.4	Une promesse, et une mise en garde.	2
1	Comprendre avant de lancer	3
1.1	Qu'est-ce qu'un modèle, au juste?	3
1.2	Qu'est-ce qu'entraîner?	3
1.3	Pourquoi on ne réentraîne pas tout : l'idée du LoRA	3
1.4	Les deux voies : partir d'un modèle, ou partir de rien	4
1.5	Généraliste ou spécialisé?	5
1.6	Le vocabulaire, en une page	5
2	Préparer sa machine	6
2.1	Ce qu'il vous faut vraiment	6
2.2	Comment la machine parle à la carte graphique	6
2.2.1	Pourquoi Vulkan, et pas DirectX	6
2.3	Installer les outils	7
2.3.1	Sur Windows	7
2.3.2	Sur Linux	7
2.3.3	Sur macOS	7
2.3.4	Sur WSL2 (Windows Subsystem for Linux)	7
2.4	Compiler les outils	8
2.5	Se procurer un modèle de départ	8
2.6	Vérifier que tout fonctionne	9
3	Le corpus : ce dont le modèle apprend.	10
3.1	Le format attendu	10
3.2	Combien d'exemples?	10
3.3	La qualité prime sur la quantité	11
3.3.1	Ce qui fait un bon exemple	11
3.3.2	Le piège du corpus fabriqué par une autre IA	11
3.4	Construire un corpus depuis votre documentation.	11
3.5	Séparer l'apprentissage du contrôle	12
3.6	Vérifier son corpus avant de lancer	12
4	Entraîner	14
4.1	La commande minimale	14
4.2	Toutes les options	14
4.3	Lire ce qui s'affiche	15
4.3.1	Au démarrage	15
4.3.2	Pendant l'entraînement	15
4.4	À quoi ressemble un apprentissage qui marche	16
4.5	Combien de temps cela va-t-il prendre?	16
4.6	Régler le taux d'apprentissage	17
4.7	Régler le rang	17
4.8	Ce que vous obtenez à la fin	18
5	Reprendre après une interruption	19
5.1	La bonne nouvelle, tout de suite	19
5.2	Pourquoi recharger les poids ne suffit pas	19

5.6	Arrêter proprement.	21
5.7	Régler la fréquence des sauvegardes.	21
5.8	Repartir vraiment de zéro	21
6	Dialoguer avec son modèle	23
6.1	Deux outils, deux usages	23
6.2	Converser : NKQwen2Chat	23
6.2.1	Les commandes	23
6.2.2	Comprendre la température	24
6.2.3	La mémoire de la conversation	24
6.3	Évaluer : NKQwen2Ask	24
6.3.1	Ce que vous voyez au chargement	25
6.3.2	Toutes les options	25
6.4	Pourquoi cet outil est lent, et pourquoi c'est voulu.	25
6.5	Comparer « avant » et « après » correctement	25
6.6	Ce que 200 pas changent réellement.	26
7	Spécialiser un modèle sur son domaine	27
7.1	Généraliste ou spécialisé : ce qui change vraiment.	27
7.2	Un cas complet, de bout en bout.	27
7.2.1	Étape 1 — écrire les questions de référence	27
7.2.2	Étape 2 — mesurer l'avant	28
7.2.3	Étape 3 — construire le corpus	28
7.2.4	Étape 4 — vérifier le corpus	28
7.2.5	Étape 5 — entraîner	29
7.2.6	Étape 6 — mesurer l'après	29
7.3	Quand ça ne marche pas.	29
7.3.1	La perte ne descend pas	29
7.3.2	La perte descend mais les réponses n'ont pas changé	29
7.3.3	Les réponses sont devenues moins bonnes	30
7.4	Aller plus loin : plusieurs domaines.	30
8	Partir de zéro	31
8.1	Pourquoi c'est rarement le bon choix	31
8.2	Pourquoi le faire quand même	31
8.3	L'outil : NKGptTrain.	32
8.4	Choisir la taille de son modèle	32
8.5	Le corpus, ici, c'est du texte brut	33
8.6	Reprendre, ici aussi.	33
8.7	Voir le modèle apprendre	34
8.8	Ce que vous ne devez pas espérer	34
9	Les fichiers : lequel contient quoi	35
9.1	Vue d'ensemble	35
9.2	Le GGUF : le modèle de base.	35
9.3	Le fichier .nkla : ce que vous avez appris	35
9.3.1	Ce que contient l'en-tête, et pourquoi	36
9.3.2	L'empreinte	36
9.4	Les checkpoints : votre filet.	36
9.4.1	Pourquoi deux fichiers	36
9.5	Que garder, que jeter.	37

9.6	Partager son travail	37
9.7	Récapitulatif des commandes	37
9.8	Pour finir	38

Avant-propos

0.1 À qui s'adresse ce cours

À quelqu'un qui n'a jamais entraîné de modèle et qui voudrait le faire *sur sa propre machine*, sans louer de serveur et sans envoyer ses données ailleurs.

Vous n'avez besoin ni de mathématiques avancées, ni d'expérience en apprentissage automatique. Vous avez besoin d'un ordinateur avec une carte graphique, d'un peu de patience, et de l'envie de comprendre ce que vous faites plutôt que de recopier des commandes.

0.2 Ce que vous saurez faire à la fin

- Expliquer, avec vos mots, ce que veut dire «entraîner un modèle».
- Préparer votre machine, sur Windows, Linux ou macOS.
- Constituer un corpus — le texte dont le modèle va apprendre.
- Lancer un entraînement qui dure plusieurs jours et qui **survit** aux coupures de courant.
- Spécialiser un modèle existant sur votre domaine à vous.
- Dialoguer avec le modèle que vous venez d'entraîner.
- Savoir quel fichier contient quoi, et lequel donner à quelqu'un.

0.3 Comment lire ce cours

Les chapitres se suivent. Chacun s'appuie sur le précédent, et chacun se termine par un exercice que vous pouvez faire en quelques minutes.

Trois encadrés reviennent tout au long du texte.

Ce qu'il faut retenir

Ce qu'il faut retenir : l'idée du passage, en deux ou trois phrases. Si vous ne deviez lire qu'une chose par section, ce serait celle-là.

Le piège

Le piège : l'erreur qui coûte une soirée. Elles sont ici parce que quelqu'un les a commises avant vous — souvent l'auteur de ce cours.

Exercice

Exercice : à faire vraiment, pas à lire. On n'apprend pas à entraîner un modèle en lisant, pas plus qu'on n'apprend à nager en regardant la piscine.

0.4 Une promesse, et une mise en garde

La promesse. Toutes les commandes de ce cours ont été exécutées sur une machine ordinaire — une carte graphique de 8 Go — avant d’être écrites ici. Les chiffres que vous lirez (durées, tailles de fichiers, valeurs de perte) sont des mesures, pas des estimations.

La mise en garde. Entraîner un modèle de sept milliards de paramètres prend du *temps*. Pas quelques minutes : plusieurs jours. Ce cours vous apprendra à travailler avec cette contrainte plutôt que contre elle — c’est d’ailleurs la raison d’être du chapitre 5, entièrement consacré à la reprise après interruption.

Ce qu’il faut retenir

Vous n’avez pas besoin d’un ordinateur exceptionnel. Vous avez besoin de savoir ce que vous faites, et d’accepter que la machine travaille pendant que vous dormez.

Comprendre avant de lancer

Ce chapitre ne contient aucune commande. Il contient les cinq idées sans lesquelles toutes les commandes des chapitres suivants ressembleraient à de la magie.

1.1 Qu'est-ce qu'un modèle, au juste ?

Imaginez un très grand tableau de nombres. Pas quelques centaines : sept milliards. Ces nombres s'appellent des **paramètres**.

Quand vous écrivez une phrase au modèle, elle traverse ce tableau. À chaque étape, des multiplications et des additions transforment votre phrase en une autre suite de nombres, jusqu'à produire une réponse : quel mot vient ensuite ?

C'est tout. Un modèle de langue est une immense machine à deviner le mot suivant. Sa culture, son style, sa capacité à raisonner — tout est rangé dans la valeur de ces sept milliards de nombres.

Ce qu'il faut retenir

Un modèle, c'est un tableau de nombres. « Entraîner » veut dire : *modifier ces nombres* pour que les réponses ressemblent davantage à celles qu'on souhaite.

1.2 Qu'est-ce qu'entraîner ?

Prenons une image. Vous apprenez à un enfant à reconnaître un chat. Vous montrez une photo, il dit « chien ». Vous corrigez : « non, chat ». Il ajuste sa manière de voir. Vingt photos plus tard, il ne se trompe plus.

L'entraînement d'un modèle suit exactement ce schéma, avec trois différences :

1. On ne corrige pas « un peu », on mesure **de combien** il s'est trompé. Ce nombre s'appelle la **perte** (*loss* en anglais). Plus il est bas, mieux c'est.
2. On ne corrige pas au hasard : on calcule, pour *chacun* des paramètres, dans quel sens le pousser pour que la perte diminue. Cet ensemble de directions s'appelle le **gradient**.
3. On recommence des milliers de fois.

Un **pas** d'entraînement, c'est donc : montrer un exemple, mesurer la perte, calculer le gradient, pousser un peu les paramètres dans le bon sens.

Ce qu'il faut retenir

La perte descend = le modèle apprend. C'est le seul chiffre à surveiller pendant les premières heures.

1.3 Pourquoi on ne réentraîne pas tout : l'idée du LoRA

Modifier sept milliards de nombres demande une carte graphique à plusieurs dizaines de milliers d'euros. Ce n'est pas votre cas, ni le mien.

L'astuce s'appelle **LoRA** — *Low-Rank Adaptation*, « adaptation de rang faible ». Elle tient en une phrase :

On **gèle** les sept milliards de paramètres d'origine, et on ajoute à côté un tout petit nombre de paramètres neufs, qui sont les seuls à apprendre.

Reprenons l'image de l'enfant. Plutôt que de lui refaire toute son éducation, on lui donne une paire de lunettes. Les lunettes sont minuscules comparées à l'enfant, mais elles suffisent à changer ce qu'il voit.

Les chiffres, sur le modèle utilisé dans ce cours :

Ce qui existe	
Paramètres du modèle d'origine (gelés)	$\approx 7\,600\,000\,000$
Ce qu'on ajoute et qui apprend	
Paramètres LoRA (rang 8)	20 185 088
Proportion	0,27 %

On entraîne donc moins de trois millièmes du modèle. Et ça marche : c'est ce que montrent les mesures du chapitre 4.

Ce qu'il faut retenir

LoRA : le modèle d'origine ne bouge pas, on lui greffe de petites pièces qui, elles, apprennent. C'est ce qui rend l'entraînement possible sur une carte de 8 Go.

1.4 Les deux voies : partir d'un modèle, ou partir de rien

Voie A — partir d'un modèle existant

Vous prenez un modèle déjà entraîné par quelqu'un d'autre — il sait déjà lire, écrire, raisonner — et vous lui apprenez *votre* domaine. C'est la voie décrite aux chapitres 3 à 7.

Coût : quelques heures à quelques jours. **Résultat** : un modèle qui parle déjà bien et qui connaît en plus votre sujet.

Voie B — partir de zéro

Vous créez un modèle vide, avec des nombres tirés au hasard, et vous lui apprenez tout : le vocabulaire, la grammaire, le sens. C'est la voie du chapitre 8.

Coût : des mois et des machines que vous n'avez pas, pour un modèle de grande taille. **Mais** c'est parfaitement faisable — et instructif — pour un petit modèle sur une tâche étroite.

Le piège

Beaucoup de débutants veulent « créer leur propre IA de zéro ». C'est compréhensible et c'est presque toujours une erreur de départ : vous dépenseriez des mois à réapprendre au modèle ce que la langue française est déjà. **Commencez par la voie A**. Le chapitre 8 vous montrera la voie B sur un problème à sa mesure.

1.5 Généraliste ou spécialisé?

Un modèle **généraliste** répond correctement à tout, sans exceller nulle part. Un modèle **spécialisé** connaît un domaine en profondeur.

La différence ne tient pas à la méthode — elle est identique — mais au **corpus** : le texte dont il apprend. Un corpus de conversations variées donne un généraliste; un corpus de documentation médicale donne un modèle médical.

Ce qu'il faut retenir

Vous ne choisissez pas « généraliste » ou « spécialisé » par un réglage. Vous le choisissez en choisissant vos textes. Le chapitre 3 est donc le plus important du cours, même s'il ne contient presque pas de commandes.

1.6 Le vocabulaire, en une page

Mot	Ce que ça veut dire
Paramètre	Un des nombres du modèle.
Perte (<i>loss</i>)	De combien le modèle s'est trompé. Plus bas = mieux.
Gradient	Dans quel sens pousser chaque paramètre pour baisser la perte.
Pas (<i>step</i>)	Un exemple montré + une correction appliquée.
Époque (<i>epoch</i>)	Un passage complet sur tout le corpus.
Corpus	L'ensemble des textes dont le modèle apprend.
Token	Un morceau de mot. « bonjour » peut valoir 2 tokens.
LoRA	Les petites pièces ajoutées qui apprennent à la place du modèle.
Rang	Taille de ces pièces. Plus grand = apprend plus, coûte plus.
Adam	La méthode qui décide de l'ampleur de chaque correction.
Checkpoint	Une photo de l'entraînement, pour pouvoir reprendre après une coupure.
Quantification	Ranger les nombres sur moins de bits pour tenir en mémoire.
VRAM	La mémoire de la carte graphique. C'est elle qui vous limite.

Exercice

Sans relire le tableau, expliquez à voix haute — vraiment à voix haute — la différence entre un *pas* et une *époque*, puis pourquoi on gèle le modèle d'origine dans LoRA. Si vous butez, relisez la section concernée : les chapitres suivants supposent ces deux idées acquises.

Préparer sa machine

2.1 Ce qu'il vous faut vraiment

Élément	Exigence réelle
Carte graphique	8 Go de mémoire vidéo suffisent pour un modèle de 7 milliards de paramètres quantifié. C'est la configuration sur laquelle toutes les mesures de ce cours ont été prises.
Mémoire vive	16 Go confortables. Le corpus complet (25 Mo de texte) ne pèse rien à côté du modèle.
Disque	15 Go : le modèle (4,4 Go), les checkpoints (231 Mo chacun, on en garde deux), et de la marge.
Système	Windows, Linux ou macOS.

Le piège

Le facteur limitant n'est pas la *vitesse* de votre carte, mais sa **mémoire**. Une carte rapide avec 4 Go ne chargera pas le modèle ; une carte lente avec 8 Go le fera tourner. Vérifiez la mémoire avant tout le reste.

2.2 Comment la machine parle à la carte graphique

Les calculs d'entraînement ne se font pas sur le processeur : ils se font sur la carte graphique, qui sait faire des millions de multiplications à la fois. Pour lui parler, il faut une **interface** — un langage commun. Nkntseu en gère quatre.

Interface	Où elle existe
Vulkan	Windows, Linux, macOS (via MoltenVK). C'est celle qu'on utilise.
DirectX 11	Windows uniquement.
DirectX 12	Windows uniquement.
OpenGL	Partout, mais la moins adaptée aux calculs.

2.2.1 Pourquoi Vulkan, et pas DirectX

Pour une raison très concrète. Le modèle contient un tenseur particulier — celui qui produit le mot final, appelé `output.weight` — qui pèse **426 Mo à lui seul**. Or DirectX 11 ne garantit que 128 Mo par ressource.

Le programme s'en occupe tout seul : il se verrouille sur Vulkan au démarrage et vous l'annonce.

Ce que vous verrez au lancement

```
1 backend compute : Vulkan
2   [NkQwen2LoraGpu] backend = Vulkan
3   [NkQwen2LoraGpu] L=28 d=3584 ffn=18944 nH=28 nKV=4 hd=128 vocab=152064
```

Le piège

Ne concluez jamais d'une mesure sans avoir lu la ligne « backend compute ». Une même commande peut tourner sur deux interfaces différentes et donner des durées sans rapport. Un résultat dont on ignore le backend est un résultat sans valeur.

2.3 Installer les outils

2.3.1 Sur Windows

Installez le **SDK Vulkan** depuis <https://vulkan.lunarg.com>. C'est tout : le reste est fourni par le dépôt.

Vérifiez ensuite que la variable `VULKAN_SDK` existe :

PowerShell

```
1 echo $env:VULKAN_SDK
```

Si la commande n'affiche rien, redémarrez la session : l'installateur pose la variable, mais les fenêtres déjà ouvertes ne la voient pas.

2.3.2 Sur Linux

Debian, Ubuntu et dérivées

```
1 sudo apt install libvulkan-dev vulkan-tools libx11-dev libxext-dev
2 vulkaninfo | head -20
```

Si `vulkaninfo` affiche le nom de votre carte, vous êtes prêt.

Le piège

Le piège du mode « headless ». Sur un serveur sans écran, on est tenté de compiler en `--linux-backend=headless`. Ne le faites pas pour l'entraînement : ce mode ne relie pas Vulkan, donc votre programme n'aurait *aucune* carte graphique à sa disposition. Compilez en `xlib` — le mode par défaut — même si vous n'affichez aucune fenêtre.

2.3.3 Sur macOS

Vulkan y passe par MoltenVK, une couche qui traduit vers Metal. Installez le SDK Vulkan pour macOS depuis le même site que pour Windows.

2.3.4 Sur WSL2 (Windows Subsystem for Linux)

Cela fonctionne, mais lisez ceci avant de vous en réjouir : WSL2 utilise *la même carte graphique* que Windows, avec une couche de traduction en plus. Vous serez donc **plus lent** qu'en

Windows natif, jamais plus rapide.

WSL2 sert à vérifier que votre travail fonctionne sous Linux sans acheter une seconde machine. Pour un entraînement de plusieurs jours, restez sur Windows natif ou sur une vraie machine Linux.

2.4 Compiler les outils

Tout se compile avec `jenga`, depuis la racine du dépôt.

Les deux programmes de ce cours

```
1 jenga build --target NKQwen2Train --config Release
2 jenga build --target NKQwen2Chat  --config Release
```

Release et non Debug : la version de mise au point est plusieurs fois plus lente, ce qui sur dix jours d'entraînement fait une différence de plusieurs semaines.

Vous devez lire, à la fin :

Compilation réussie

```
1 Projects Built: 21/21
2 Status:        SUCCESS
```

Le piège

Si vous compilez *tout* le dépôt avec `jenga build` sans `--target`, vous verrez peut-être échouer un projet de démonstration sans rapport avec l'IA. Ce n'est pas votre faute et cela n'empêche rien : compilez toujours la cible qui vous intéresse.

2.5 Se procurer un modèle de départ

Il vous faut un modèle déjà entraîné, au format **GGUF**. Le plus simple est de passer par Ollama :

Télécharger Qwen2.5 7B

```
1 ollama pull qwen2.5:7b
```

Le fichier atterrit dans un dossier de blobs, sous un nom qui est son empreinte :

Windows	%USERPROFILE%\ollama\models\blobs\
Linux, macOS	~/.ollama/models/blobs/

Repérez le fichier de 4,4 Go : c'est le modèle. Les petits fichiers à côté sont des métadonnées.

Le piège

Vérifiez que vous avez le bon fichier. Un dossier de blobs contient souvent plusieurs modèles de taille voisine. Se tromper de blob donne un modèle qui répond n'importe quoi, avec des jetons spéciaux inattendus — et l'on cherche l'erreur pendant des heures dans son code alors qu'elle est dans le chemin. Notez le chemin exact de *votre* modèle, une bonne fois.

2.6 Vérifier que tout fonctionne

Avant d'entraîner quoi que ce soit, faites parler le modèle. Si cette étape échoue, inutile d'aller plus loin.

Windows — PowerShell

```
1 .\Build\Bin\Release-Windows\NKQwen2Chat\NKQwen2Chat.exe `
2 "$env:USERPROFILE\.ollama\models\blobs\sha256-VOTRE_BLOB"
```

Linux et macOS

```
1 ./Build/Bin/Release-Linux/NKQwen2Chat/NKQwen2Chat \
2 ~/.ollama/models/blobs/sha256-VOTRE_BLOB
```

Le modèle met une dizaine de secondes à se charger, puis vous rend la main. Posez une question, appuyez sur Entrée, et regardez la réponse s'écrire mot à mot.

Ce qu'il faut retenir

Trois vérifications, dans cet ordre, avant tout entraînement : la ligne « backend compute » dit **Vulkan**; le modèle se charge sans erreur de mémoire; il répond quelque chose de sensé. Si les trois passent, votre machine est prête.

Exercice

Chargez le modèle et posez-lui trois questions : une factuelle (« Quelle est la capitale du Cameroun ? »), une de raisonnement (« Si tous les A sont B et que X est un A, que peut-on dire de X ? »), une créative (« Écris deux phrases sur la pluie »). Notez les réponses dans un fichier. Vous les comparerez à celles d'après l'entraînement, au chapitre 7 — et c'est la seule façon honnête de juger si votre entraînement a servi à quelque chose.

Le corpus : ce dont le modèle apprend

Voici le chapitre le plus important du cours. Un entraînement parfaitement conduit sur un mauvais corpus donne un mauvais modèle; il n'existe aucun réglage pour rattraper cela.

3.1 Le format attendu

Un corpus est un simple fichier texte. Chaque exemple tient sur deux lignes : une question, une réponse.

corpus.txt — le format, en entier

```
1 Question: Quelle est la capitale du Cameroun ?
2 Reponse: La capitale politique du Cameroun est Yaoundé.
3
4 Question: Comment convertir des degrés Celsius en Fahrenheit ?
5 Reponse: On multiplie par 1,8 puis on ajoute 32.
6
7 Question: Qu'est-ce qu'un syllogisme ?
8 Reponse: Un raisonnement à trois propositions où la conclusion découle
9 des deux premières.
```

Trois règles, et c'est tout :

- La ligne de question commence exactement par Question: (avec l'espace).
- La ligne de réponse commence exactement par Reponse: — **sans accent**, c'est le format historique du dépôt.
- Les lignes vides sont ignorées : mettez-en pour aérer.

Le piège

Une question sans réponse est *silencieusement ignorée*. Si votre corpus de 10 000 exemples n'en donne que 4 000 au chargement, cherchez les Reponse: mal orthographiés — un accent de trop suffit.

3.2 Combien d'exemples?

Taille	Ce que vous pouvez en attendre
100 – 500	Change le style des réponses : leur longueur, leur ton, leur mise en forme. Ne change pas les connaissances.
1 000 – 10 000	Le modèle assimile un vocabulaire et des tournures propres à votre domaine.
50 000 et plus	Le modèle intègre réellement le contenu de votre domaine.

Le corpus utilisé pour les mesures de ce cours contient **100 017 paires** et pèse 25 Mo.

Ce qu'il faut retenir

Peu d'exemples changent la *forme*, beaucoup d'exemples changent le *fond*. Si votre modèle « répond mieux » après 200 pas, il a surtout appris à mieux se présenter — ce qui est déjà utile, mais ne le confondez pas avec du savoir.

3.3 La qualité prime sur la quantité

3.3.1 Ce qui fait un bon exemple

- **Une réponse exacte.** Le modèle ne vérifie rien : il imite. Une erreur dans le corpus devient une erreur apprise.
- **Une réponse complète.** « Oui » n'apprend rien. « Oui, parce que... » apprend à justifier.
- **Une formulation naturelle.** Écrivez la réponse comme vous voudriez la lire.
- **De la variété dans les questions.** Si vos mille questions commencent toutes par « Qu'est-ce que », le modèle ne saura répondre qu'à celles-là.

3.3.2 Le piège du corpus fabriqué par une autre IA

Il est tentant de demander à un modèle existant de générer des milliers de paires question/réponse. C'est rapide, et c'est un piège.

Un modèle qui apprend des réponses d'un autre modèle apprend surtout son **style** : sa façon de tourner les phrases, ses tics, ses formules. Il n'apprend pas de connaissances nouvelles — l'autre modèle ne connaissait pas votre domaine non plus.

Le piège

Pour *spécialiser* un modèle sur votre domaine, le corpus doit venir de **vos sources réelles** : votre documentation, vos archives, vos manuels. Un corpus généré par une IA transmet un style, pas un savoir.

3.4 Construire un corpus depuis votre documentation

La méthode qui marche, en trois temps :

1. **Découper.** Prenez vos documents et coupez-les en passages qui se suffisent à eux-mêmes — un paragraphe, une section courte.
2. **Interroger.** Pour chaque passage, écrivez la ou les questions auxquelles il répond. C'est le travail le plus long, et le plus utile.
3. **Répondre.** La réponse est le passage lui-même, reformulé pour qu'il se tienne seul.

Un passage de documentation devenu deux exemples

- ```
1 Question: Comment reprendre un entraînement interrompu avec NKQwen2Train ?
2 Reponse: Relancez exactement la même commande. Le programme cherche les
3 checkpoints, choisit le plus avancé qui soit intact, et reprend au pas
4 et à l'exemple où il s'était arrêté.
5
6 Question: Que se passe-t-il si un checkpoint est corrompu ?
7 Reponse: Il est détecté par son empreinte, ignoré, et la reprise se fait
```

```
8 depuis le second checkpoint. C'est la raison pour laquelle deux fichiers
9 sont conservés en alternance.
```

#### Ce qu'il faut retenir

Écrire les questions est le vrai travail. C'est aussi ce qui rend le corpus irremplaçable : il contient votre manière d'interroger votre domaine, et aucune IA ne peut la deviner à votre place.

### 3.5 Séparer l'apprentissage du contrôle

Voici une erreur qui trompe beaucoup de débutants, et qui a une parade simple.

Si vous mesurez les progrès du modèle sur les exemples qu'il vient d'apprendre, vous mesurerez sa **mémoire**, pas sa compréhension. Un élève à qui l'on repose exactement les questions de l'exercice qu'il vient de corriger aura toujours l'air brillant.

La parade : mettez de côté quelques dizaines d'exemples que le modèle ne verra **jamais** pendant l'entraînement. Ils serviront à vérifier.

- Si la perte d'apprentissage descend *et* celle de contrôle descend : le modèle apprend vraiment. **C'est ce qu'on veut**.
- Si la perte d'apprentissage descend mais que celle de contrôle stagne ou remonte : le modèle récite. On appelle cela le **surapprentissage**.

NKQwen2Train s'en charge : il évalue périodiquement un exemple pris loin de la position courante dans le corpus, et affiche sa perte à part.

#### Ce qu'il faut retenir

Deux pertes, pas une. Celle qui compte est celle mesurée sur ce que le modèle n'a jamais vu.

### 3.6 Vérifier son corpus avant de lancer

Un entraînement de dix jours sur un corpus mal formé est dix jours perdus. Trois minutes de vérification les économisent.

#### Compter les questions et les réponses

```
1 # Linux, macOS
2 grep -c "^Question: " corpus.txt
3 grep -c "^Reponse: " corpus.txt
4
5 # Windows -- PowerShell
6 (Select-String -Path corpus.txt -Pattern '^Question: ').Count
7 (Select-String -Path corpus.txt -Pattern '^Reponse: ').Count
```

Les deux nombres doivent être **égaux**, ou presque. Un écart important signale des paires incomplètes.

Puis lancez l'entraînement sur deux pas seulement, juste pour lire le compte :

#### Vérification en trente secondes

```
1 NKQwen2Train --gguf=<modele> --corpus=corpus.txt --steps=2 --fresh
```

La ligne corpus : N paires vous dit combien d'exemples ont réellement été retenus. Si ce nombre vous surprend, corrigez le corpus avant d'aller plus loin.

#### Exercice

Constituez un corpus de vingt paires sur un sujet que vous connaissez bien — votre métier, une passion, votre ville. Écrivez-les à la main : vingt paires, c'est une demi-heure. Vérifiez ensuite avec les commandes ci-dessus que les vingt sont bien lues. Ce corpus vous servira aux chapitres suivants.

# Entraîner

## 4.1 La commande minimale

Deux informations suffisent : le modèle de départ et le corpus.

### Windows — PowerShell

```
1 .\Build\Bin\Release-Windows\NKQwen2Train\NKQwen2Train.exe `
2 --gguf="$env:USERPROFILE\.ollama\models\blobs\sha256-VOTRE_BLOB" `
3 --corpus="C:\mes-donnees\corpus.txt" `
4 --out="C:\mes-donnees\mon-modele"
```

### Linux et macOS

```
1 ./Build/Bin/Release-Linux/NKQwen2Train/NKQwen2Train \
2 --gguf=~/.ollama/models/blobs/sha256-VOTRE_BLOB \
3 --corpus=~/.mes-donnees/corpus.txt \
4 --out=~/.mes-donnees/mon-modele
```

C'est la même commande partout : seuls les chemins changent.

## 4.2 Toutes les options

| Option         | Défaut     | À quoi ça sert                                                          |
|----------------|------------|-------------------------------------------------------------------------|
| --gguf=        | —          | Le modèle de départ. <b>Obligatoire.</b>                                |
| --corpus=      | —          | Le fichier de textes. <b>Obligatoire.</b>                               |
| --out=         | qwen2_lora | Préfixe des fichiers produits.                                          |
| --lr=          | 1e-4       | Taux d'apprentissage : l'ampleur de chaque correction.                  |
| --steps=       | 0          | Nombre de pas sur ce lancement. 0 signifie « jusqu'au bout du corpus ». |
| --save-every=  | 200        | Sauvegarde tous les $n$ pas.                                            |
| --rank=        | 8          | Taille des pièces LoRA.                                                 |
| --max-seq=     | 128        | Longueur maximale d'un exemple, en tokens. Les plus longs sont sautés.  |
| --valid-every= | 500        | Contrôle tous les $n$ pas. 0 pour ne jamais contrôler.                  |
| --fresh        | —          | Ignorer les checkpoints et repartir de zéro.                            |

## Ce qu'il faut retenir

Tout se règle par la ligne de commande. **Changer de corpus, de taux d'apprentissage ou de durée ne demande jamais de recompiler.** Recompiler ne redevient nécessaire que si l'on modifie le programme lui-même.

## 4.3 Lire ce qui s'affiche

### 4.3.1 Au démarrage

#### Sortie réelle, machine à 8 Go

```
1 === NKQwen2Train – entraînement LoRA reprenable ===
2
3 backend compute : Vulkan
4 [NKQwen2LoraGpu] L=28 d=3584 ffn=18944 nH=28 nKV=4 hd=128 vocab=152064
5 couche 1/28 chargée (0.7 s, 0.31 Go)
6 couche 28/28 chargée (10.1 s, 3.83 Go)
7 lm_head = output.weight (Q6_K, 152064 x 3584, 426 Mo)
8 adaptateurs : 20185088 paramètres entraînaables (77 Mo) + 231 Mo d'état Adam
9 modèle chargé en 13.4 s
10 corpus : 100017 paires
11 aucun checkpoint exploitable – départ de zéro
```

Ligne par ligne :

- **backend compute : Vulkan** — la vérification du chapitre 2. Si cette ligne dit autre chose, arrêtez-vous.
- **L=28 d=3584** — 28 couches, 3584 nombres pour représenter un token. C'est la carte d'identité du modèle.
- **3,83 Go** — la place prise sur votre carte graphique. Sur 8 Go, il reste de quoi travailler.
- **20185088 paramètres entraînaables** — les fameux 0,27 %.
- **231 Mo d'état Adam** — la mémoire de l'optimiseur; on y revient au chapitre 5.
- **100017 paires** — combien d'exemples ont été lus. Comparez à ce que vous attendiez.

### 4.3.2 Pendant l'entraînement

#### Une ligne toutes les dix étapes

```
1 pas 10 | perte 2.2903 (moy. 1.8772) | 8.46 s/pas | exemple 10/100017
```

|                   |                                                               |
|-------------------|---------------------------------------------------------------|
| pas 10            | Nombre total de pas depuis le tout début, reprises comprises. |
| perte 2.2903      | L'erreur sur cet exemple. Elle saute beaucoup : c'est normal. |
| (moy. 1.8772)     | La moyenne. <b>C'est celle-ci qu'on surveille.</b>            |
| 8.46 s/pas        | Le rythme.                                                    |
| exemple 10/100017 | Où l'on en est dans le corpus.                                |

### Le piège

Ne tirez aucune conclusion de la perte d'un exemple isolé. Elle dépend d'abord de la difficulté de cet exemple-là. Une phrase courte et banale donne une perte basse, une phrase technique une perte haute — sans que le modèle ait progressé ou régressé entre les deux. **Seule la moyenne, sur des centaines de pas, raconte quelque chose.**

## 4.4 À quoi ressemble un apprentissage qui marche

Voici des mesures réelles, prises sur 200 pas :

| Pas | Perte d'apprentissage | Perte de contrôle |
|-----|-----------------------|-------------------|
| 0   | —                     | 1,6613            |
| 1   | 2,2370                | 1,6212            |
| 25  | 1,2378                | 0,8662            |
| 50  | 1,8171                | 0,7430            |
| 100 | 1,2766                | 0,6655            |
| 150 | 0,9647                | 0,6580            |
| 200 | 0,8639                | <b>0,6292</b>     |

Deux choses à observer.

**La colonne du milieu monte et descend.** Au pas 50 elle vaut 1,82, plus qu'au pas 25. Ce n'est pas une régression : c'est un exemple plus difficile.

**La colonne de droite descend franchement,** de 1,66 à 0,63. Comme elle est mesurée sur des exemples jamais vus, cette baisse est un vrai progrès et non de la récitation.

### Ce qu'il faut retenir

Un bon entraînement se reconnaît à ceci : la perte de contrôle descend régulièrement. Si elle stagne pendant des centaines de pas, quelque chose ne va pas — le plus souvent le corpus, parfois le taux d'apprentissage.

## 4.5 Combien de temps cela va-t-il prendre ?

La question que tout le monde pose. La réponse mesurée :

|                                        |                           |
|----------------------------------------|---------------------------|
| Durée d'un pas (moyenne)               | 9,15 s                    |
| Durée d'un pas (observé)               | de 8,5 à 14,2 s           |
| 1 000 pas                              | ≈ 2 h 30                  |
| 10 000 pas                             | ≈ 25 h                    |
| Une époque complète (100 017 exemples) | <b>254 h = 10,6 jours</b> |

Oui : dix jours et demi pour un seul passage sur le corpus entier.

Ce chiffre n'est pas un défaut à corriger, c'est la réalité d'un modèle de sept milliards de paramètres sur une carte grand public. Trois façons de faire avec :

1. **Laisser tourner.** C'est pour cela que la reprise existe (chapitre 5). La machine travaille la nuit, vous l'arrêtez quand vous en avez besoin, vous la relancez ensuite.
2. **Réduire le corpus à ce qui compte.** 10 000 exemples bien choisis valent mieux que 100 000 exemples médiocres — et prennent un jour au lieu de dix.
3. **Écourter les exemples.** Baisser `--max-seq` accélère chaque pas, au prix des exemples longs qui seront sautés.

#### Le piège

La durée d'un pas dépend de la **longueur** de l'exemple. Un corpus de questions courtes ira nettement plus vite qu'un corpus de longs paragraphes. Les 10,6 jours sont une moyenne sur ce corpus-là, pas une loi universelle : mesurez sur le vôtre avec `--steps=20`.

## 4.6 Régler le taux d'apprentissage

C'est l'ampleur de chaque correction. Trop petit, le modèle n'apprend rien ; trop grand, il part dans le décor.

- 
- 1e-3 Rapide et risqué. La perte peut exploser.
  - 1e-4 **Le défaut.** Fonctionne dans la grande majorité des cas. Commencez par là.
  - 1e-5 Prudent. Pour affiner un modèle déjà bien adapté.
- 

#### Le piège

Si votre perte grimpe au lieu de descendre, ou affiche nan, votre taux est trop grand. Divisez-le par dix et relancez avec `--fresh` : les paramètres déjà abîmés ne se récupèrent pas.

## 4.7 Régler le rang

Le rang fixe la taille des pièces ajoutées.

| Rang | Paramètres | Quand le choisir                                  |
|------|------------|---------------------------------------------------|
| 4    | ≈ 10 M     | Adapter le style, peu de mémoire disponible.      |
| 8    | 20 M       | <b>Le défaut.</b> Le bon compromis.               |
| 16   | ≈ 40 M     | Domaine riche, beaucoup d'exemples.               |
| 32   | ≈ 80 M     | Rarement utile ; à réserver aux très gros corpus. |

#### Le piège

Le rang doit être **identique** entre l'entraînement et la reprise. Reprendre un checkpoint de rang 8 en demandant un rang 16 est refusé — et c'est heureux : les formes ne correspondraient pas, et le modèle rechargé serait faux.

## 4.8 Ce que vous obtenez à la fin

### Fin d'un entraînement

```
1 checkpoint final : mon-modele_ckpt0.nk1a (231.0 Mo)
2 adaptateurs livrables : mon-modele.nk1a (77.0 Mo, 20185088 paramètres)
3
4 6 pas en 76 s (12.73 s/pas) | perte moyenne 1.9923 | total cumulé : 6 pas
```

Deux sortes de fichiers, à ne pas confondre :

| Fichier                     | Taille | À quoi il sert                                                    |
|-----------------------------|--------|-------------------------------------------------------------------|
| _ckpt0.nk1a,<br>_ckpt1.nk1a | 231 Mo | <b>Reprendre</b> un entraînement. Contient tout l'état.           |
| .nk1a                       | 77 Mo  | <b>Utiliser</b> le modèle. C'est ce fichier-là que vous partagez. |

### Ce qu'il faut retenir

Le fichier à donner, c'est le .nk1a de 77 Mo. Les checkpoints ne servent qu'à vous, sur votre machine, pour continuer là où vous vous êtes arrêté.

### Exercice

Reprenez le corpus de vingt paires du chapitre 3 et lancez : `--steps=20 --save-every=10 --valid-every=0`. Comptez le temps qu'un pas prend *sur votre machine* et multipliez par le nombre d'exemples de votre vrai corpus. Vous saurez à quoi vous engager avant de vous engager.

# Reprendre après une interruption

Sur dix jours d'entraînement, une coupure n'est pas une éventualité : c'est une certitude. Panne de courant, mise à jour qui redémarre la machine, besoin soudain de la carte graphique pour autre chose. Ce chapitre explique comment tout cela devient sans conséquence.

## 5.1 La bonne nouvelle, tout de suite

Reprendre : la même commande, exactement

```
1 NKQwen2Train --gguf=<modele> --corpus=<corpus.txt> --out=mon-modele
```

C'est tout. Aucune option de reprise à ajouter. Le programme cherche les checkpoints, prend le plus avancé qui soit intact, et repart.

Ce qu'il affiche alors

```
1 REPRISE : pas 6, exemple 6/100017, époque 0
```

Ce qu'il faut retenir

Relancer la même commande suffit. Si vous voyez « REPRISE », le travail précédent est retrouvé. Si vous voyez « aucun checkpoint exploitable », il repart de zéro — et il faut comprendre pourquoi avant de le laisser tourner.

## 5.2 Pourquoi recharger les poids ne suffit pas

Voici l'idée la plus subtile du cours. Prenez le temps.

Naïvement, on croirait qu'il suffit de sauvegarder les paramètres appris. C'est faux, et une reprise faite ainsi **diverge dès le premier pas**.

La raison tient à la méthode qui décide de l'ampleur des corrections, appelée **Adam**. Adam ne se contente pas du gradient courant : il garde une mémoire de ceux d'avant, sous la forme de deux moyennes appelées *moments*. C'est ce qui lui permet d'avancer avec régularité plutôt que de zigzaguer.

Or ces moyennes démarrent à zéro, donc sont trop faibles au début. Adam corrige ce défaut par un facteur qui dépend du **numéro du pas** : très fort au pas 1, négligeable au pas 1000.

Maintenant, imaginez une reprise qui recharge les poids mais oublie le compteur :

- Les moments sont grands (mille pas d'histoire);
- mais le compteur est retombé à zéro, donc Adam applique la correction du *tout premier pas*;
- il en résulte un pas gigantesque, et l'entraînement part en vrille.

Dix jours de calcul détruits en une correction.

### Ce qu'il faut retenir

Un checkpoint doit contenir **quatre** choses, pas une : les paramètres, les deux moments d'Adam, le numéro du pas, et la position dans le corpus. C'est la différence entre reprendre et recommencer en pire.

## 5.3 Ce que contient réellement un checkpoint

|                                                |                                                                  |
|------------------------------------------------|------------------------------------------------------------------|
| Les paramètres LoRA                            | Ce que le modèle a appris.                                       |
| Les deux moments d'Adam                        | La mémoire de l'optimiseur — deux fois la taille des paramètres. |
| Le numéro du pas                               | Pour que la correction de biais reste juste.                     |
| La position dans le corpus                     | Pour ne pas réapprendre les mêmes exemples.                      |
| Le numéro d'époque                             | Combien de fois le corpus a été parcouru.                        |
| L'état du tirage aléatoire                     | Pour que l'ordre reste reproductible.                            |
| Le taux d'apprentissage et les réglages d'Adam | Pour reprendre à l'identique.                                    |

D'où la taille : **231 Mo** contre 77 Mo pour les seuls adaptateurs. Les moments pèsent deux fois les paramètres.

## 5.4 Écrire sans jamais rien détruire

Un checkpoint de 231 Mo met environ une seconde à s'écrire. Que se passe-t-il si le courant saute pendant cette seconde ?

Si l'on écrivait directement sur le fichier existant : on perdrait à la fois l'ancien (déjà écrasé) et le nouveau (incomplet). **La sauvegarde aurait détruit ce qu'elle devait protéger.**

Deux protections travaillent ensemble.

### 5.4.1 L'écriture passe par un fichier temporaire

Le programme écrit dans `mon-modele_ckpt0.nk1a.tmp`, puis — une fois le fichier complet — le **renomme**. Le renommage est quasi instantané, là où l'écriture dure une seconde.

### 5.4.2 Deux checkpoints en alternance

Les sauvegardes vont tour à tour dans `_ckpt0` et `_ckpt1`. À tout instant, l'un des deux est une génération complète et saine.

### Ce qu'il faut retenir

Fichier temporaire puis renommage, et deux fichiers en alternance. Ces deux mesures ensemble font qu'aucune coupure, à aucun moment, ne peut vous laisser sans checkpoint valide.

## 5.5 Quand un fichier s'abîme

Un disque peut abîmer un fichier sans prévenir. Chaque checkpoint porte donc une **empreinte** calculée sur l'ensemble de son contenu. Au chargement, elle est recalculée et comparée.

Voici une expérience réelle. Un checkpoint a été délibérément abîmé — 64 octets écrasés en plein milieu, exactement ce que ferait un secteur défectueux — puis l’entraînement relancé :

#### Sortie réelle après corruption volontaire

```
1 checkpoint mon-modele_ckpt0.nkla ignoré :
2 NkLoraGpuLoadNKLA : empreinte FNV incorrecte – fichier corrompu
3 REPRISE : pas 10, exemple 10/100017, époque 0
```

Corruption détectée, fichier écarté, reprise depuis l’autre — **sans perdre un seul pas**.

#### Le piège

Un fichier corrompu est **refusé**, jamais chargé «au mieux». Charger des paramètres à moitié faux donnerait un modèle silencieusement dégradé : il répondrait, mais mal, et vous cherchiez pendant des jours une erreur qui serait en réalité une corruption de disque.

## 5.6 Arrêter proprement

Pour interrompre volontairement : **Ctrl+C**.

Le programme ne s’arrête pas net. Il termine le pas en cours, écrit un checkpoint, puis quitte :

#### Arrêt volontaire

```
1 interruption demandée – écriture de l'état avant de quitter
2 checkpoint final : mon-modele_ckpt1.nkla (231.0 Mo)
3 adaptateurs livrables : mon-modele.nkla (77.0 Mo, 20185088 paramètres)
```

Vous récupérez donc la carte graphique immédiatement, sans rien perdre.

## 5.7 Régler la fréquence des sauvegardes

|                   |                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------|
| --save-every=50   | Sauvegarde fréquente. Vous perdez au plus 50 pas ( $\approx 8$ min), mais vous écrivez 231 Mo toutes les 8 minutes. |
| --save-every=200  | <b>Le défaut.</b> Environ toutes les 30 minutes. Bon compromis.                                                     |
| --save-every=1000 | Peu d’écritures, mais une coupure peut coûter 2 h 30 de calcul.                                                     |

#### Ce qu’il faut retenir

Le bon réglage répond à une seule question : *combien de temps de calcul acceptez-vous de perdre ?* Une sauvegarde coûte une seconde ; une reprise manquée coûte des heures.

## 5.8 Repartir vraiment de zéro

Si vous voulez ignorer tout ce qui existe et recommencer :

### Ignorer les checkpoints

```
1 NKQwen2Train --gguf=<modele> --corpus=<corpus.txt> --out=mon-modele --fresh
```

### Le piège

--fresh n'efface pas les checkpoints : il les ignore. Ils seront écrasés à la première sauvegarde du nouvel entraînement. Si vous teniez à l'ancien, **copiez-le ailleurs avant**, ou changez de --out.

### Exercice

Lancez un entraînement avec --steps=10 --save-every=5. Laissez-le finir. Relancez la même commande : vous devez lire « REPRISE : pas 10 ». Puis relancez avec --fresh : vous devez lire « les checkpoints existants sont ignorés ». Ces deux messages sont ceux que vous lirez pendant des jours — apprenez à les distinguer d'un coup d'œil.

# Dialoguer avec son modèle

Entraîner un modèle sans jamais lui parler, c'est cuisiner sans goûter. Ce chapitre présente les deux outils de dialogue et, surtout, explique lequel choisir selon ce que vous cherchez à savoir.

## 6.1 Deux outils, deux usages

|                            | NKQwen2Chat         | NKQwen2Ask                   |
|----------------------------|---------------------|------------------------------|
| Adaptateurs                | <b>non</b>          | <b>oui</b>                   |
| Mémoire de la conversation | oui (KV-cache)      | non                          |
| Affichage                  | mot à mot           | réponse d'un bloc            |
| Vitesse                    | ≈ 0,5 s par mot     | ≈ 3 s par mot                |
| Sert à                     | converser, explorer | <b>juger un entraînement</b> |

### Ce qu'il faut retenir

NKQwen2Chat pour *discuter*. NKQwen2Ask pour *évaluer*. Le second est lent, et c'est un choix : il emprunte exactement le même chemin de calcul que l'entraînement, adaptateurs compris.

## 6.2 Converser : NKQwen2Chat

### Lancer une conversation

```
1 NKQwen2Chat <chemin_du_modele>
```

Le modèle se charge en une dizaine de secondes, puis vous rend la main. Écrivez, validez, la réponse s'écrit mot à mot.

### 6.2.1 Les commandes

|             |                                              |
|-------------|----------------------------------------------|
| /aide       | La liste des commandes.                      |
| /oublie     | Vide la mémoire de la conversation.          |
| /temp <v>   | 0 = toujours la même réponse; 0,7 = créatif. |
| /tokens <n> | Longueur maximale d'une réponse.             |
| /etat       | Contexte utilisé, mémoire, vitesse.          |
| /quitte     | Sortir.                                      |

## 6.2.2 Comprendre la température

C'est le seul réglage qui change vraiment la personnalité des réponses.

- **0** — le modèle prend toujours le mot le plus probable. Même question, même réponse, indéfiniment. **C'est le réglage pour comparer** : sans lui, vous ne sauriez pas si une différence vient de votre entraînement ou du hasard.
- **0,7** — le modèle pioche parmi les mots probables. Réponses plus vivantes, mais différentes à chaque fois.
- **au-delà de 1** — il devient franchement fantaisiste.

### Le piège

Pour juger un entraînement, mettez la température à **0**. À 0,7, deux réponses différentes ne prouvent rien : le hasard suffit à les expliquer.

## 6.2.3 La mémoire de la conversation

Le modèle se souvient de ce qui précède, dans la limite de son contexte. Vous pouvez donc enchaîner :

### Une conversation qui se tient

```
1 > Qu'est-ce qu'un syllogisme ?
2 Un raisonnement à trois propositions où la conclusion découle des deux
3 premières.
4
5 > Donne-m'en un exemple.
6 Tous les hommes sont mortels. Socrate est un homme. Donc Socrate est mortel.
```

La seconde question n'a de sens que grâce à la première : c'est la mémoire qui opère.

### Le piège

Cette mémoire a une limite. Quand la conversation devient trop longue, le contexte est vidé et le modèle « oublie » d'un coup. Utilisez /oublie délibérément entre deux sujets sans rapport : vous éviterez que le début d'une conversation ne parasite la suite.

## 6.3 Évaluer : NKQwen2Ask

C'est l'outil qui charge vos adaptateurs.

### Sans adaptateurs — la référence « avant »

```
1 NKQwen2Ask --gguf=<modele> --question="Quelle est la capitale du Cameroun ?"
```

### Avec vos adaptateurs — le « après »

```
1 NKQwen2Ask --gguf=<modele> --adapters=mon-modele.nkla \
2 --question="Quelle est la capitale du Cameroun ?"
```

Sans --question, il ouvre un dialogue : posez vos questions les unes après les autres, /quitte pour sortir.

### 6.3.1 Ce que vous voyez au chargement

#### Sortie réelle, adaptateurs appliqués

```
1 backend compute : Vulkan
2 modèle chargé en 7.8 s
3 adaptateurs appliqués : mon-modele.nk1a
4 20185088 paramètres · rang 8 · alpha 16 · 28 couches × 7 projections
5
6 > Quelle est la capitale du Cameroun ?
7 (calcul en cours – sans KV-cache, compter plusieurs dizaines de secondes)
8
9 La capitale du Cameroun est Yaoundé.
10
11 [13 tokens en 44.4 s]
```

Lisez la ligne des adaptateurs : elle confirme que le fichier a été accepté, et avec quel rang. Si le rang ne correspond pas à celui de l'entraînement, le chargement est **refusé** — et c'est heureux.

### 6.3.2 Toutes les options

| Option      | Défaut | Rôle                               |
|-------------|--------|------------------------------------|
| --gguf=     | —      | Le modèle. <b>Obligatoire</b> .    |
| --adapters= | aucun  | Le .nk1a à appliquer.              |
| --tokens=   | 48     | Longueur maximale de la réponse.   |
| --rank=     | 8      | Doit être celui de l'entraînement. |
| --max-seq=  | 256    | Contexte maximal.                  |
| --question= | —      | Poser une question et sortir.      |

## 6.4 Pourquoi cet outil est lent, et pourquoi c'est voulu

NKQwen2Ask met environ 3 secondes par mot, contre 0,5 pour NKQwen2Chat. Ce n'est pas un défaut : c'est une conséquence assumée.

NKQwen2Chat garde en mémoire les calculs des mots précédents — le KV-cache — et n'a donc à calculer que le mot nouveau. NKQwen2Ask recalcule toute la séquence à chaque mot, parce qu'il emprunte le **chemin exact de l'entraînement**, adaptateurs compris.

L'intérêt est là : si la réponse change entre « avant » et « après », c'est votre entraînement qui l'a changée, et rien d'autre. Avec deux chemins de calcul différents, vous ne sauriez jamais si la différence vient de l'apprentissage ou de l'implémentation.

#### Ce qu'il faut retenir

Un outil lent qui compare honnêtement vaut mieux qu'un outil rapide dont on ne peut rien conclure. Utilisez NKQwen2Ask pour juger, pas pour bavarder.

## 6.5 Comparer « avant » et « après » correctement

Voici la méthode, en quatre temps.

**1. Préparez vos questions à l'avance.** Cinq à dix, écrites dans un fichier, avant tout entraînement. Les inventer après biaiserait votre jugement.

2. **Mesurez l'avant.** Sans --adapters. Notez les réponses.
3. **Entraînez.**
4. **Mesurez l'après,** avec les mêmes questions, exactement.

### Le piège

**N'attendez pas de miracle après quelques pas.** Voici une mesure réelle, après 8 pas d'entraînement :

|       |                                      |
|-------|--------------------------------------|
| Avant | La capitale du Cameroun est Yaoundé. |
| Après | La capitale du Cameroun est Yaoundé. |

Strictement identique — et c'est **normal**. Huit pas modifient les paramètres de façon infime. Il faut des centaines de pas pour voir le style bouger, des milliers pour le fond. Une réponse inchangée après quelques pas ne veut pas dire que l'entraînement ne fonctionne pas : c'est la perte, et elle seule, qui vous le dit à ce stade.

## 6.6 Ce que 200 pas changent réellement

Après 200 pas sur un corpus de logique, sur les mêmes questions :

---

« Qu'est-ce qu'un syllogisme ? »

---

|       |                                                                                          |
|-------|------------------------------------------------------------------------------------------|
| Avant | une forme de raisonnement logique qui consiste en une argumentation composée de trois... |
| Après | une forme de raisonnement logique qui consiste à déduire une conclusion à partir...      |

---

La réponse a changé : elle est plus directe. Mais elle n'est pas *plus juste* — le modèle savait déjà. Deux cents pas déplacent la **manière de dire**, pas le savoir.

### Ce qu'il faut retenir

Ne confondez pas « la réponse a changé » avec « le modèle a appris quelque chose ». Pour du savoir nouveau, il faut un corpus qui le contienne et des milliers de pas — c'est l'objet du chapitre suivant.

### Exercice

Reprenez les trois questions notées à l'exercice du chapitre 2. Posez-les à NKQwen2Ask sans adaptateurs, notez les réponses. Vous tenez votre référence « avant » — gardez ce fichier, il vous servira à chaque entraînement futur.

# Spécialiser un modèle sur son domaine

Les chapitres précédents ont montré les outils. Celui-ci les met bout à bout sur un cas complet, du corpus vide au modèle qui répond sur votre sujet.

## 7.1 Généraliste ou spécialisé : ce qui change vraiment

Rien, dans les commandes. Tout, dans le corpus.

| Corpus                       | Modèle obtenu                                                                   |
|------------------------------|---------------------------------------------------------------------------------|
| Conversations variées        | Généraliste : répond correctement à tout, sans exceller nulle part.             |
| Documentation d'un logiciel  | Répond sur ce logiciel : ses commandes, ses messages d'erreur, ses conventions. |
| Textes juridiques d'un pays  | Emploie le vocabulaire et les tournures de ce droit-là.                         |
| Archives d'un service client | Répond dans le ton de la maison, sur ses produits.                              |

### Ce qu'il faut retenir

Vous ne réglez pas la spécialisation, vous la *nourrissez*. Le temps que vous passerez à choisir vos textes comptera davantage que tous les réglages réunis.

## 7.2 Un cas complet, de bout en bout

Prenons un exemple concret : spécialiser un modèle sur la documentation de votre propre logiciel, pour qu'il réponde aux questions de vos utilisateurs.

### 7.2.1 Étape 1 — écrire les questions de référence

**Avant tout**, écrivez les questions qui serviront à juger. Dix suffisent.

#### questions.txt

- 1 Comment reprendre un entraînement interrompu ?
- 2 Que se passe-t-il si un checkpoint est corrompu ?
- 3 Quelle est la différence entre un checkpoint et un fichier d'adaptateurs ?
- 4 Pourquoi utiliser Vulkan plutôt que DirectX ?
- 5 Combien de temps prend une époque complète ?

### Le piège

Écrire ces questions *après* avoir vu les réponses du modèle est le plus sûr moyen de se convaincre d'un progrès qui n'existe pas. On choisit alors, sans le vouloir, les questions où le modèle brille.

## 7.2.2 Étape 2 — mesurer l'avant

### La référence

```
1 NKQwen2Ask --gguf=<modele> --question="Comment reprendre un entraînement
2 interrompu ?" --tokens=64
```

Sans corpus spécialisé, le modèle répondra quelque chose de générique et de plausible — souvent faux dans les détails. C'est exactement ce qu'on veut constater, et conserver.

## 7.2.3 Étape 3 — construire le corpus

Reprenez la méthode du chapitre 3 : découper, interroger, répondre.

### corpus-doc.txt — un extrait

```
1 Question: Comment reprendre un entraînement interrompu ?
2 Reponse: Relancez exactement la même commande. Le programme cherche les
3 checkpoints, retient le plus avancé qui soit intact, et repart du pas et
4 de l'exemple où il s'était arrêté.
5
6 Question: Que se passe-t-il si un checkpoint est corrompu ?
7 Reponse: Son empreinte ne correspond plus, il est refusé et ignoré. La
8 reprise se fait depuis le second checkpoint, sans perdre de pas.
9
10 Question: Quelle est la différence entre un checkpoint et un fichier
11 d'adaptateurs ?
12 Reponse: Le checkpoint pèse 231 Mo et sert à reprendre un entraînement :
13 il contient les moments d'Adam et la position dans le corpus. Le fichier
14 d'adaptateurs pèse 77 Mo, ne contient que ce qui a été appris, et sert à
15 utiliser ou partager le modèle.
```

**Combien en faut-il ?** Pour une documentation, comptez trois à cinq questions par page. Une documentation de 200 pages donne donc de 600 à 1 000 paires — de quoi occuper la machine une journée.

## 7.2.4 Étape 4 — vérifier le corpus

### Trente secondes bien investies

```
1 NKQwen2Train --gguf=<modele> --corpus=corpus-doc.txt --steps=2 --fresh
```

Lisez la ligne `corpus` :  $N$  paires. Si  $N$  ne correspond pas à ce que vous avez écrit, corrigez maintenant, pas dans dix heures.

## 7.2.5 Étape 5 — entraîner

### L'entraînement réel

```
1 NKQwen2Train --gguf=<modele> \
2 --corpus=corpus-doc.txt \
3 --out=modele-doc \
4 --lr=1e-4 \
5 --save-every=100 \
6 --valid-every=200
```

Laissez tourner. Surveillez deux choses, et deux seulement :

- la **perte moyenne** doit descendre;
- la **perte de contrôle** doit descendre aussi.

## 7.2.6 Étape 6 — mesurer l'après

Mêmes questions, mêmes réglages, adaptateurs en plus :

### Le verdict

```
1 NKQwen2Ask --gguf=<modele> --adapters=modele-doc.nkla \
2 --question="Comment reprendre un entraînement interrompu ?" \
3 --tokens=64
```

### Ce qu'il faut retenir

Le seul jugement qui vaille est celui-ci : sur vos questions, préparées *avant*, la réponse est-elle devenue plus juste ? Pas plus longue, pas plus jolie : plus juste.

## 7.3 Quand ça ne marche pas

### 7.3.1 La perte ne descend pas

| Cause probable                   | Vérification                                                                                         |
|----------------------------------|------------------------------------------------------------------------------------------------------|
| Corpus mal formé                 | La ligne corpus : $N$ paires donne un $N$ bien inférieur au nombre attendu.                          |
| Taux d'apprentissage trop faible | Essayez <code>--lr=3e-4</code> .                                                                     |
| Taux trop fort                   | La perte oscille violemment ou affiche nan. Divisez par dix, et relancez avec <code>--fresh</code> . |
| Exemples trop longs              | Ils sont sautés en silence. Augmentez <code>--max-seq</code> , ou raccourcissez vos réponses.        |

### 7.3.2 La perte descend mais les réponses n'ont pas changé

C'est le cas le plus fréquent, et le plus mal interprété. Deux explications possibles :

1. **Vous n'avez pas fait assez de pas.** Quelques centaines déplacent le style ; il en faut des milliers pour le fond.

2. **Le modèle savait déjà.** Si vos questions portent sur des généralités, il y répondait correctement avant. Interrogez-le sur ce que *seule* votre documentation contient — un nom de commande, un message d'erreur, une convention interne.

### 7.3.3 Les réponses sont devenues moins bonnes

C'est le **surapprentissage** : le modèle a tellement bien mémorisé votre corpus qu'il a désappris à répondre au reste. Signe caractéristique : la perte d'apprentissage continue de descendre pendant que celle de contrôle remonte.

#### Le piège

Le remède n'est pas de « moins entraîner » au hasard. Prenez le checkpoint *d'avant* la remontée — c'est précisément pour cela qu'on sauvegarde régulièrement. Copiez de temps en temps un checkpoint ailleurs, avec le numéro du pas dans son nom : vous pourrez revenir en arrière.

## 7.4 Aller plus loin : plusieurs domaines

Rien ne vous empêche d'entraîner plusieurs jeux d'adaptateurs sur le même modèle de base :

|                       |                                      |
|-----------------------|--------------------------------------|
| modele-doc.nkla       | répond sur votre documentation       |
| modele-juridique.nkla | répond sur vos contrats              |
| modele-support.nkla   | répond dans le ton du service client |

Chacun pèse 77 Mo, et le modèle de base de 4,4 Go reste partagé. Vous changez de spécialité en changeant un argument :

#### Un modèle, trois spécialités

```
1 NKQwen2Ask --gguf=<modele> --adapters=modele-doc.nkla --question="..."
2 NKQwen2Ask --gguf=<modele> --adapters=modele-juridique.nkla --question="..."
```

#### Ce qu'il faut retenir

C'est là tout l'intérêt du LoRA : le gros fichier ne bouge jamais, seules les petites pièces changent. Distribuer une spécialité coûte 77 Mo, pas 4,4 Go.

#### Exercice

Choisissez un domaine que vous connaissez et dont le modèle ne peut rien savoir — le règlement intérieur de votre école, la documentation d'un projet à vous. Écrivez cent paires. Entraînez mille pas. Comparez avant et après sur cinq questions préparées d'avance. C'est le projet qui transforme ce cours en compétence.

## Partir de zéro

Jusqu'ici, nous avons toujours pris un modèle existant pour lui greffer des pièces. Ce chapitre montre l'autre voie : construire un modèle vide et lui apprendre la langue elle-même.

### 8.1 Pourquoi c'est rarement le bon choix

Soyons francs avant de commencer.

|                 | Affiner (LoRA)                                    | Partir de zéro                                |
|-----------------|---------------------------------------------------|-----------------------------------------------|
| Point de départ | Un modèle qui sait déjà lire et écrire            | Des nombres tirés au hasard                   |
| Corpus          | Quelques milliers d'exemples                      | Des milliards de mots                         |
| Durée           | Heures à jours                                    | Semaines à mois                               |
| Matériel        | Une carte de 8 Go                                 | Plusieurs cartes professionnelles             |
| Résultat        | Un modèle qui parle bien et connaît votre domaine | Un modèle qui parle mal, sauf effort colossal |

#### Le piège

«Je veux créer ma propre IA de zéro» est un souhait naturel et presque toujours une erreur de départ. Vous dépenseriez des mois à réapprendre à une machine ce que la langue française est — travail déjà fait, et mieux fait, par ceux dont c'est le métier.

### 8.2 Pourquoi le faire quand même

Trois raisons valables, et elles ne sont pas économiques :

1. **Comprendre.** Voir un modèle passer du charabia à des mots français apprend plus que dix articles. C'est la meilleure raison.
2. **Une langue peu dotée.** Si votre langue n'est présente dans aucun modèle existant, il n'y a rien à affiner. C'est le cas de nombreuses langues africaines.
3. **Un problème étroit.** Pour une tâche très spécifique — reconnaître une forme, classer des messages courts — un petit modèle entraîné de zéro peut battre un grand modèle généraliste, et tourner mille fois plus vite.

#### Ce qu'il faut retenir

Partez de zéro pour *apprendre*, ou parce que rien n'existe dans votre langue. Jamais pour «faire mieux» qu'un modèle existant sur une tâche générale : le rapport de forces n'est pas en votre faveur.

## 8.3 L'outil : NKGptTrain

Le dépôt fournit un GPT complet, entraînable de zéro, avec son propre découpage en tokens.

### Compiler

```
1 jenga build --target NKGptTrain --config Release
```

Il se configure par un **fichier**, ce qui présente un avantage décisif sur une longue série d'essais : le fichier se garde, se relit, se compare, se range à côté du modèle produit. Six mois plus tard, vous saurez encore avec quels réglages ce modèle a été obtenu.

### mon-gpt.cfg

```
1 # --- Corpus ---
2 NK_GPT_DIR=./mes-textes
3 NK_GPT_CHARS=1200000
4
5 # --- Taille du modèle ---
6 NK_GPT_I=256 # longueur de contexte, en tokens
7 NK_GPT_D=384 # largeur : nombres par token
8 NK_GPT_H=6 # têtes d'attention
9 NK_GPT_L=6 # couches
10 NK_GPT_B=16 # exemples traités ensemble
11 NK_GPT_MERGES=4000 # taille du vocabulaire
12
13 # --- Entraînement ---
14 NK_GPT_STEPS=20000
15 NK_GPT_LR=3e-4
16 NK_GPT_WARMUP=500
17 NK_GPT_SAVEEVERY=1000
18
19 # --- Contrôle ---
20 NK_GPT_VALFRAC=0.05
21 NK_GPT_VALEVERY=500
22
23 # --- Fichiers ---
24 NK_GPT_SAVE=mon-gpt.nkgp
```

### Lancer

```
1 NK_GPT_CONFIG=mon-gpt.cfg NKGptTrain
```

## 8.4 Choisir la taille de son modèle

Quatre nombres déterminent tout. Voici comment les lire.

- 
- T Contexte** : combien de tokens le modèle voit d'un coup. À 256, il ne peut pas relier le début d'un long paragraphe à sa fin.
  - D Largeur** : combien de nombres décrivent un token. C'est la richesse de sa représentation.
  - H Têtes** : combien de façons différentes de relier les mots entre eux. **D doit être divisible par H.**
  - L Profondeur** : combien de couches. C'est ce qui permet le raisonnement en plusieurs temps.
- 

|       | T   | D   | H  | L  | Ce qu'on obtient                                                       |
|-------|-----|-----|----|----|------------------------------------------------------------------------|
| Jouet | 128 | 128 | 4  | 4  | Du français qui ressemble à du français, sans sens. Quelques heures.   |
| Petit | 256 | 384 | 6  | 6  | Des phrases correctes, un sens fragile. <b>Le bon point de départ.</b> |
| Moyen | 512 | 768 | 12 | 12 | Des paragraphes tenables. Plusieurs jours.                             |

---

#### Le piège

La tentation est de viser grand tout de suite. Faites l'inverse : commencez par le modèle «jouet». S'il ne produit rien de lisible après quelques heures, c'est votre corpus ou vos réglages qu'il faut corriger — et vous l'aurez appris en heures plutôt qu'en semaines.

## 8.5 Le corpus, ici, c'est du texte brut

Pas de Question: ni de Reponse: : le modèle apprend à *continuer* du texte, pas à répondre. Donnez-lui des fichiers texte, il les lira à la suite.

#### Organisation

```
1 mes-textes/
2 romans.txt
3 articles.txt
4 documentation.txt
```

**Combien?** Le réglage `NK_GPT_CHARS` fixe le budget de caractères. Un million de caractères ( $\approx 200\,000$  mots, deux romans) suffit pour un modèle jouet. En dessous, le modèle apprendra le texte par cœur au lieu d'apprendre la langue.

## 8.6 Reprendre, ici aussi

Le principe du chapitre 5 vaut pour cet outil :

### Reprendre un GPT

- 1 NK\_GPT\_RESUME=1
- 2 NK\_GPT\_LOAD=mon-gpt.nkgp

Les checkpoints portent l'extension `.nkgp` et sont écrits tous les `NK_GPT_SAVEEVERY` pas.

## 8.7 Voir le modèle apprendre

C'est le moment qui justifie tout le chapitre. Générez du texte à intervalles réguliers, et regardez :

| Pas    | Ce que le modèle produit                                                           |
|--------|------------------------------------------------------------------------------------|
| 0      | Des caractères au hasard.                                                          |
| 500    | Des suites de lettres qui ont la <i>forme</i> de mots, des espaces au bon endroit. |
| 2 000  | De vrais mots français, mais assemblés sans grammaire.                             |
| 10 000 | Des phrases grammaticalement correctes, au sens vacillant.                         |
| 50 000 | Des paragraphes cohérents sur quelques lignes.                                     |

### Ce qu'il faut retenir

Cette progression est la meilleure leçon d'apprentissage automatique qui soit. Le modèle apprend la *forme* avant le *sens* — les lettres, puis les mots, puis la grammaire, puis, très lentement, le sens. Aucun cours ne fait comprendre cela aussi bien que de le voir.

## 8.8 Ce que vous ne devez pas espérer

Un modèle entraîné chez vous sur quelques millions de caractères produira des phrases françaises. Il ne saura pas répondre à vos questions, ne connaîtra aucun fait, et se contredira d'une phrase à l'autre.

Ce n'est pas un échec : c'est ce que produit cette quantité de données. Les modèles auxquels vous êtes habitué ont vu des milliers de milliards de mots.

### Ce qu'il faut retenir

Partir de zéro apprend *comment* un modèle apprend. Pour un modèle *utile*, revenez au chapitre 4 : affiner un modèle existant reste, et de loin, le meilleur usage de votre machine et de votre temps.

### Exercice

Entraînez un modèle jouet ( $T = 128$ ,  $D = 128$ ,  $H = 4$ ,  $L = 4$ ) sur un livre du domaine public. Faites-lui produire dix lignes de texte au pas 100, au pas 1 000, puis au pas 10 000. Collez les trois résultats côte à côte : vous tiendrez, en une page, une démonstration de ce qu'est l'apprentissage.

# Les fichiers : lequel contient quoi

Vous allez accumuler des fichiers de tailles très différentes. Ce chapitre dit lequel garder, lequel partager, et lequel effacer sans regret.

## 9.1 Vue d'ensemble

| Extension     | Taille   | Contenu                                                           |
|---------------|----------|-------------------------------------------------------------------|
| .gguf         | 4,4 Go   | Le modèle de base, quantifié. Vient d'ailleurs, ne change jamais. |
| .nkla         | 77 Mo    | Les adaptateurs entraînés. <b>C'est ce qu'on partage.</b>         |
| _ckpt0/1.nkla | 231 Mo   | Checkpoint de reprise. Personnel, temporaire.                     |
| .nkgp         | variable | Checkpoint d'un GPT entraîné de zéro.                             |

## 9.2 Le GGUF : le modèle de base

C'est le gros fichier téléchargé au chapitre 2. Il contient les sept milliards de paramètres, **quantifiés** — c'est-à-dire rangés sur moins de bits que leur précision d'origine, pour tenir en mémoire.

|             |                                                                                                                             |
|-------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Q4_K</b> | Environ 4 bits par nombre. La majorité des tenseurs.                                                                        |
| <b>Q6_K</b> | Environ 6 bits. Réserve aux tenseurs les plus sensibles — dont <code>output.weight</code> , celui qui choisit le mot final. |
| <b>F32</b>  | 32 bits, pleine précision. Les normalisations, petites et critiques.                                                        |

### Ce qu'il faut retenir

Sans quantification, ce modèle occuperait plus de 28 Go et ne tiendrait sur aucune carte grand public. C'est elle qui rend tout ce cours possible.

**Ce fichier n'est jamais modifié par l'entraînement.** C'est le principe même du LoRA : le socle est gelé. Vous pouvez le mettre en lecture seule.

## 9.3 Le fichier .nkla : ce que vous avez appris

|            |                                  |
|------------|----------------------------------|
| Taille     | 77 Mo                            |
| Paramètres | 20 185 088                       |
| Contenu    | les pièces LoRA, et rien d'autre |

C'est le résultat de votre travail. Pour l'utiliser, il faut aussi le GGUF d'origine : le .nk1a seul ne sert à rien, comme une paire de lunettes sans personne pour les porter.

### 9.3.1 Ce que contient l'en-tête, et pourquoi

Le fichier commence par sa carte d'identité : nombre de couches, de projections, rang, alpha, et les dimensions du modèle auquel il est destiné.

Ces dimensions sont redondantes avec le GGUF — c'est voulu. Elles permettent de **refuser** un adaptateur qu'on tenterait d'appliquer au mauvais modèle, plutôt que de l'appliquer de travers et de produire un modèle silencieusement faux.

### 9.3.2 L'empreinte

Le fichier se termine par une empreinte calculée sur tout son contenu. Au chargement, elle est recalculée et comparée. Si elle diffère, le fichier est **refusé**.

#### Le piège

Un adaptateur corrompu et chargé « au mieux » donnerait un modèle qui répond encore — mais mal, de façon subtile. Vous cherchiez l'erreur dans votre corpus pendant des jours. Un refus franc est toujours préférable à une dégradation silencieuse.

## 9.4 Les checkpoints : votre filet

|                          |                 |
|--------------------------|-----------------|
| Les paramètres LoRA      | 77 Mo           |
| Le premier moment d'Adam | 77 Mo           |
| Le second moment d'Adam  | 77 Mo           |
| L'état d'entraînement    | quelques octets |
| <b>Total</b>             | <b>231 Mo</b>   |

Les moments pèsent deux fois les paramètres : c'est toute la différence entre 77 et 231 Mo.

L'état d'entraînement, minuscule, est pourtant ce qui rend la reprise correcte : numéro du pas, position dans le corpus, numéro d'époque, état du tirage aléatoire, taux d'apprentissage et réglages d'Adam.

#### Ce qu'il faut retenir

Un checkpoint est un fichier *personnel et temporaire*. On ne le partage pas : votre interlocuteur n'a que faire de vos moments d'Adam. Il veut le .nk1a de 77 Mo.

### 9.4.1 Pourquoi deux fichiers

\_ckpt0 et \_ckpt1 sont écrits en alternance. À tout instant, l'un des deux est une génération complète et saine. Si une coupure survient pendant l'écriture, ou si un disque abîme un fichier, l'autre reste.

#### Le repli en action

```
1 checkpoint mon-modele_ckpt0.nk1a ignoré :
2 empreinte FNV incorrecte – fichier corrompu
3 REPRISE : pas 10, exemple 10/100017, époque 0
```

## 9.5 Que garder, que jeter

| Fichier   | Après l'entraînement                        | Pourquoi                                                                             |
|-----------|---------------------------------------------|--------------------------------------------------------------------------------------|
| .gguf     | <b>Garder</b>                               | Nécessaire pour utiliser vos adaptateurs.                                            |
| .nk1a     | <b>Garder, sauvegarder</b>                  | C'est votre travail. 77 Mo se copient partout.                                       |
| _ckpt0/1  | Jeter si vous ne comp-<br>tez pas continuer | 462 Mo à deux, et ils ne servent qu'à reprendre.                                     |
| Le corpus | <b>Garder</b>                               | Pour réentraîner, corriger, augmenter. Souvent plus précieux que le modèle lui-même. |

### Le piège

**Sauvegardez votre corpus avant votre modèle.** Un modèle perdu se réentraîne en quelques jours de machine; un corpus perdu, ce sont des semaines de travail humain. C'est lui, le bien rare.

## 9.6 Partager son travail

Pour donner votre modèle spécialisé à quelqu'un, deux choses :

1. Le **nom exact** du modèle de base (par exemple qwen2.5:7b) — il le téléchargera lui-même.
2. Votre fichier .nk1a de 77 Mo.

Ajoutez-y, par honnêteté, le **rang** utilisé : sans lui, votre interlocuteur devra deviner l'argument --rank.

### Ce qu'il tapera

```
1 ollama pull qwen2.5:7b
2 NKQwen2Ask --gguf=<son_blob> --adapters=votre-modele.nk1a --rank=8
```

## 9.7 Récapitulatif des commandes

| Ce que vous voulez        | La commande                                       |
|---------------------------|---------------------------------------------------|
| Vérifier que tout marche  | NKQwen2Chat <modele>                              |
| Mesurer avant d'entraîner | NKQwen2Ask --gguf=<m> --question="..."            |
| Vérifier son corpus       | NKQwen2Train ... --steps=2 --fresh                |
| Entraîner                 | NKQwen2Train --gguf=<m> --corpus=<c><br>--out=<o> |
| Reprendre                 | la <i>même</i> commande                           |
| Recommencer de zéro       | la même commande + --fresh                        |
| Juger le résultat         | NKQwen2Ask --gguf=<m> --adapters=<o>.nk1a         |

| Ce que vous voulez       | La commande                      |
|--------------------------|----------------------------------|
| Entraîner un GPT de zéro | NK_GPT_CONFIG=<f>.cfg NKGptTrain |

## 9.8 Pour finir

Vous savez maintenant préparer une machine, constituer un corpus, entraîner un modèle pendant des jours sans craindre les coupures, spécialiser ce modèle sur votre domaine, et juger honnêtement s'il s'est amélioré.

Le reste est une affaire de patience et de corpus. Ce sont les deux seules choses qu'aucun outil ne fera à votre place.

### Ce qu'il faut retenir

Trois idées à emporter :

1. Le corpus décide de tout. Les réglages ne rattrapent jamais de mauvais textes.
2. La perte de contrôle est le seul juge fiable pendant l'entraînement; vos questions préparées d'avance sont le seul juge fiable après.
3. Sur dix jours, une coupure est certaine. C'est pourquoi la reprise n'est pas un confort mais la condition même du travail.

### Exercice

Le projet complet, à faire en une semaine.

**Jour 1.** Choisissez un domaine que vous connaissez et dont aucun modèle ne peut rien savoir. Écrivez dix questions de référence, et mesurez l'avant.

**Jours 2 et 3.** Constituez un corpus de 300 à 500 paires depuis vos sources réelles.

**Jour 4.** Vérifiez le corpus, lancez l'entraînement, laissez tourner.

**Jour 5.** Mesurez l'après sur les dix questions. Notez ce qui a changé et ce qui n'a pas changé.

**Jours 6 et 7.** Corrigez ce que la comparaison a révélé — presque toujours le corpus — et recommencez.

Ce cycle est le métier. Tout le reste n'est que de l'outillage.